

Tech Tour

Nordic TechTour 2026

Workshop – Bluetooth LE, LTE Cat 1bis, Power Management

Christian Kurz

11/2026

NORDIC SEMICONDUCTOR

Welcome & Big Picture

HANDS-ON WORKSHOP

NORDIC SEMICONDUCTOR

Workshop Solution – Step by Step

We will build a complete application by adding the following components step by step.

1 Sensor



Start with a simulated sensor to collect data.
At the end, replace it with a real sensor.

2 Bluetooth



Use Bluetooth to send a notification of the remaining battery charge.

3 PMIC



Use the PMIC for power supply and to measure battery parameters (voltage, current, temperature).

Report state of charge and state of health using the Nordic fuel gauge algorithm.

4 Cellular

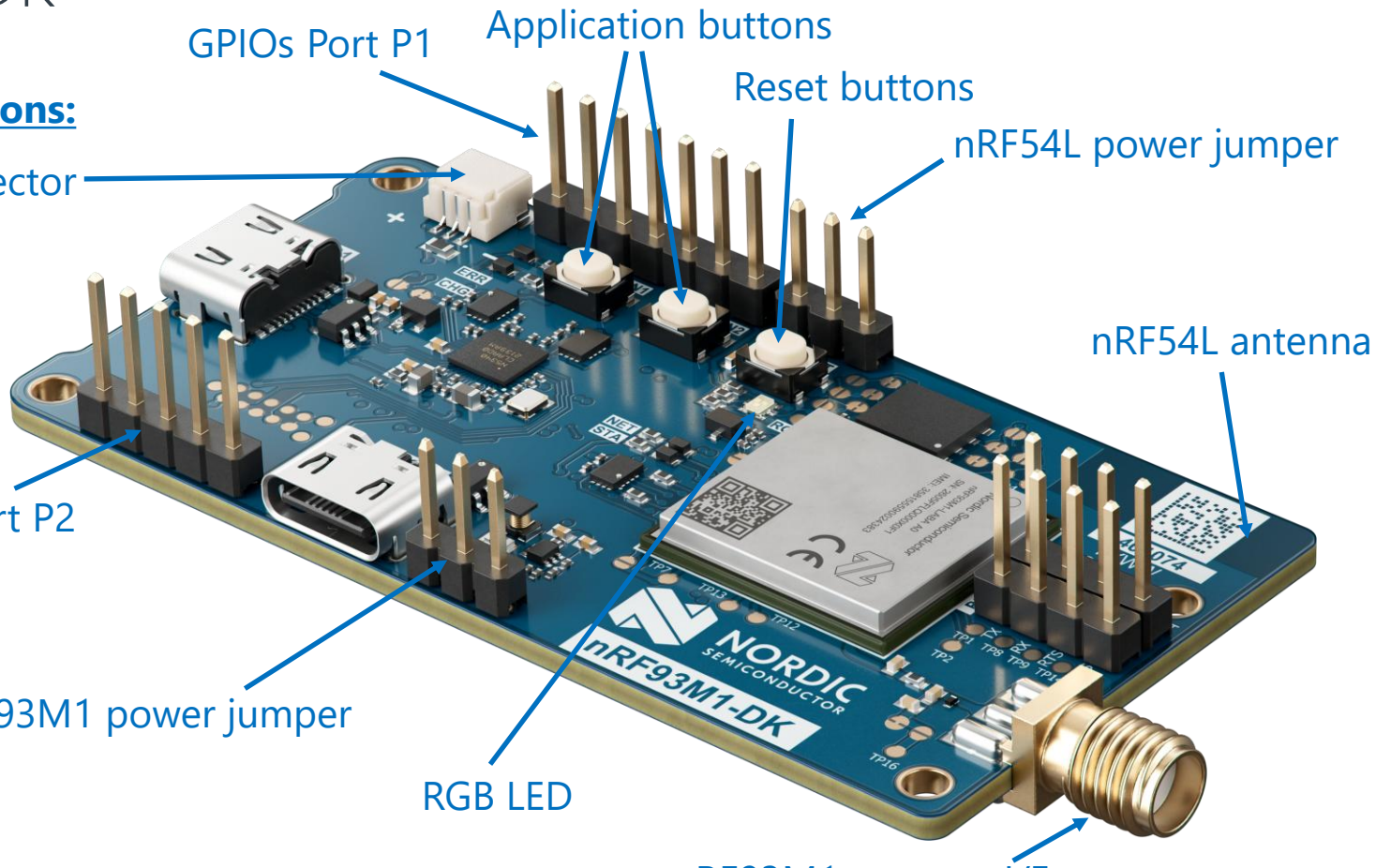


Use cellular to get location information and upload sensor data to the cloud.



nRF93M1 DK

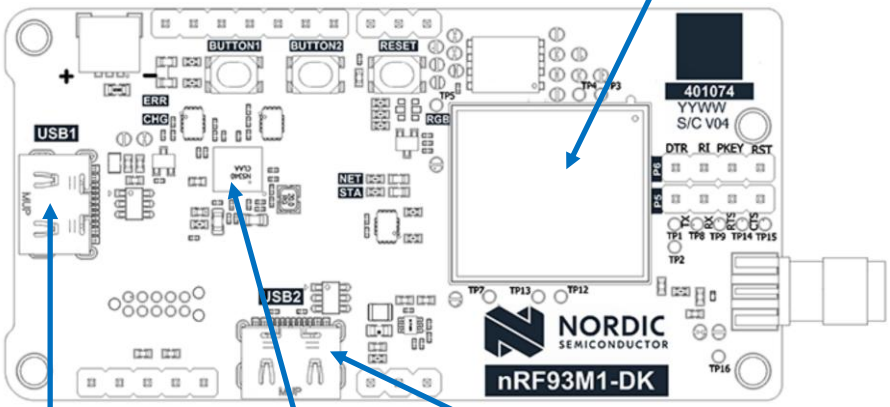
Power Supply Options:



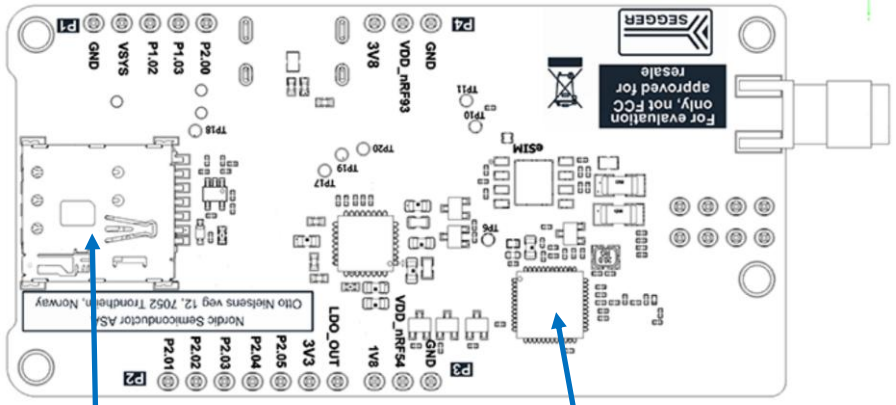
Copyright © 2026 Nordic Semiconductor. All rights reserved

nRF93M1 DK

- nRF93M1 DK front view

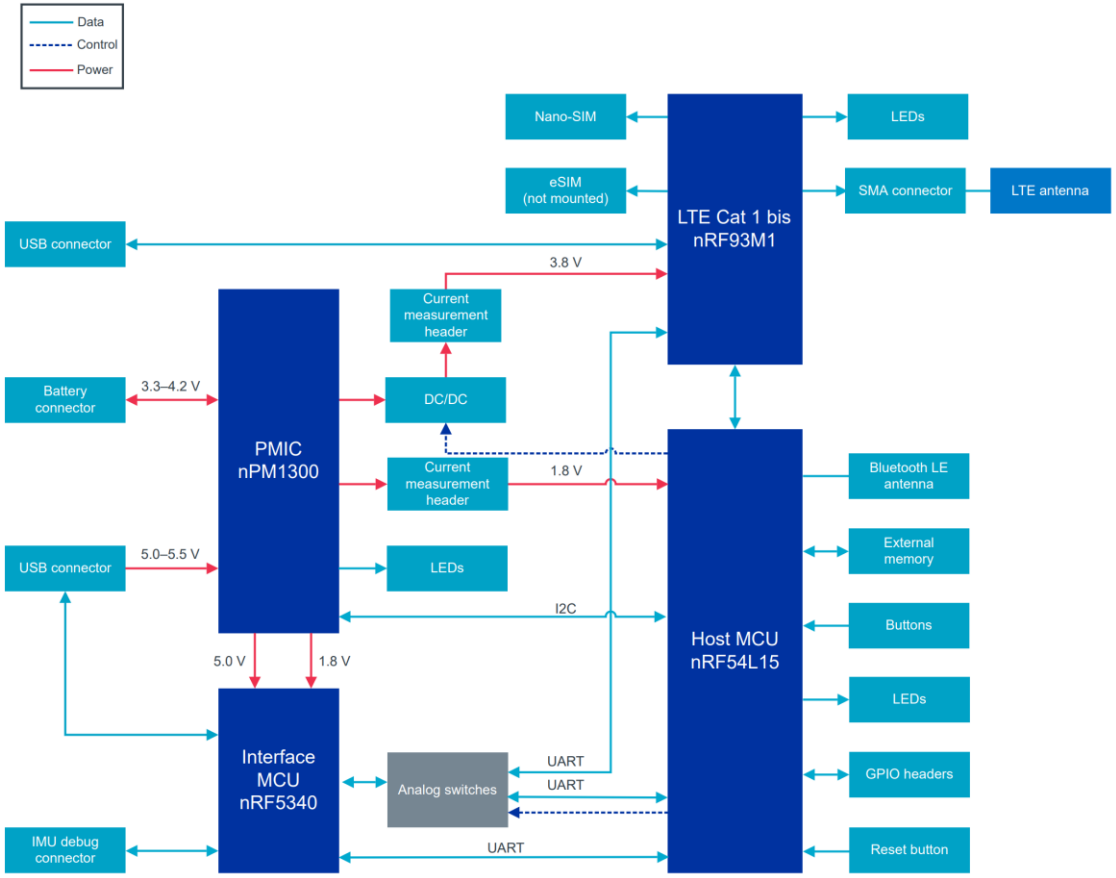


- nRF93M1 DK back view



Copyright © 2026 Nordic Semiconductor. All rights reserved

nRF93M1-DK Block Diagram



Copyright © 2026 Nordic Semiconductor.

Command Line vs. GUI

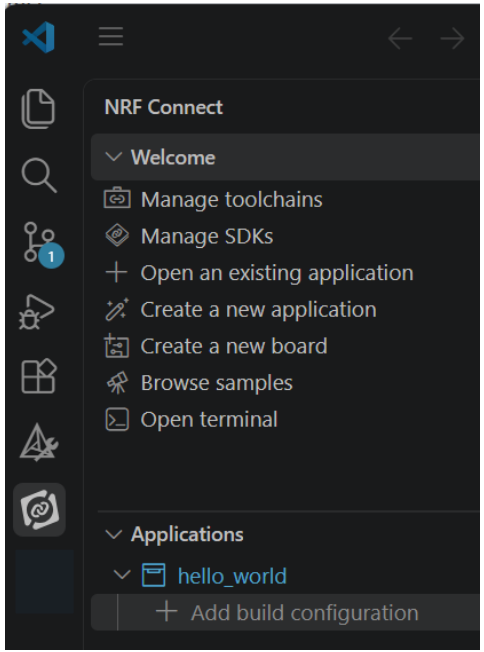
Explorer

Search

Debugger

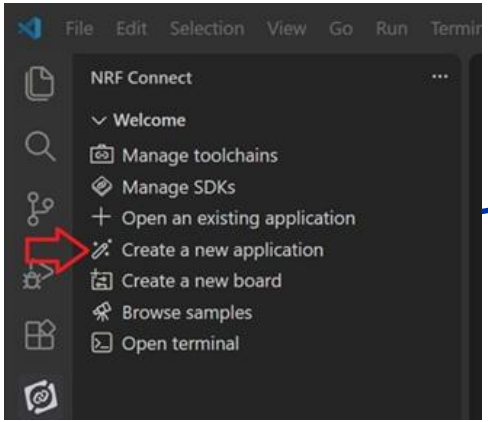
Extensions: Marketplace

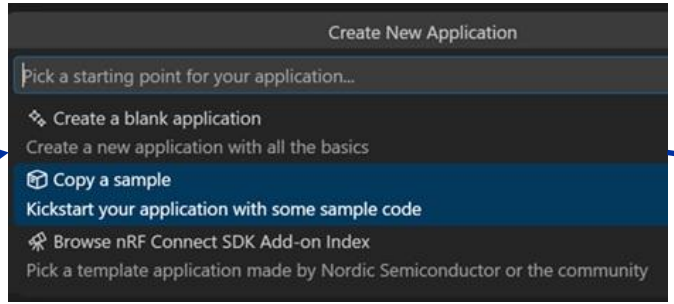
nRF Connect

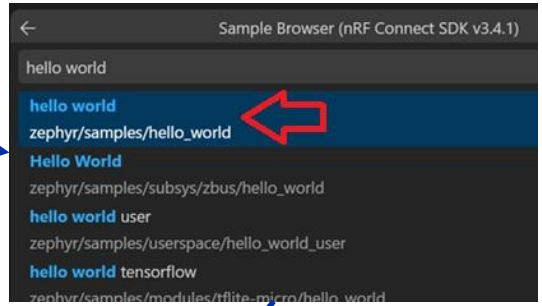


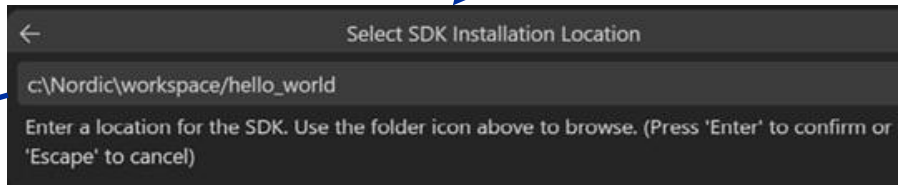
Copyright © 2026 Nordic Semiconductor. All rights reserved

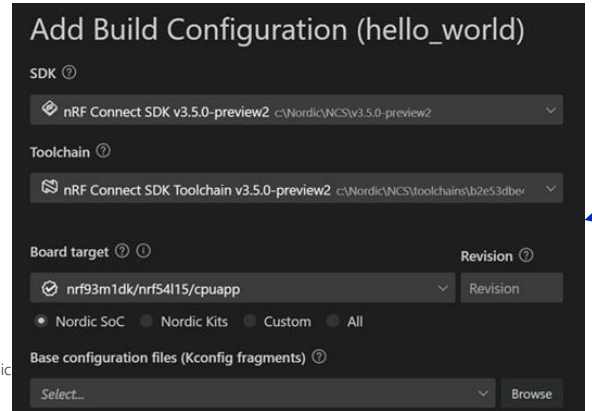
Copy a Sample





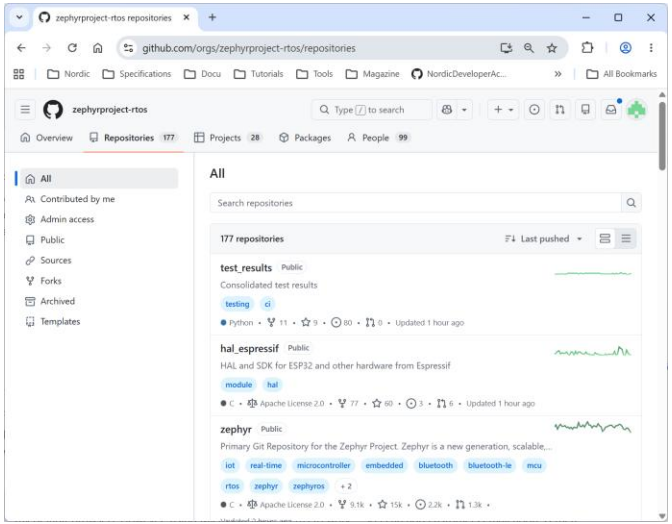


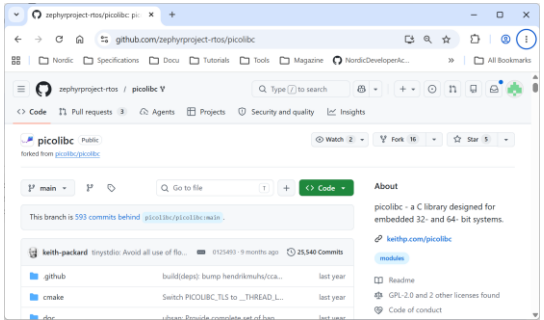


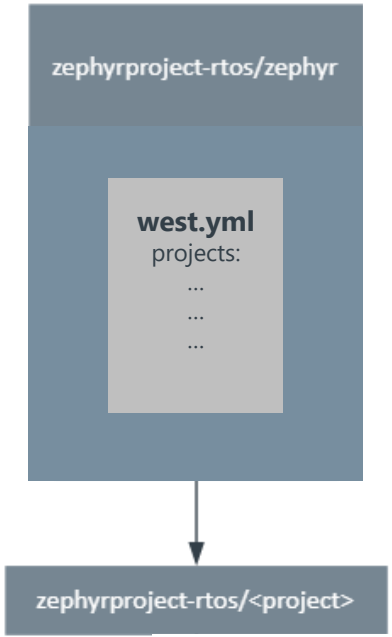


Copyright © 2026 Nordic

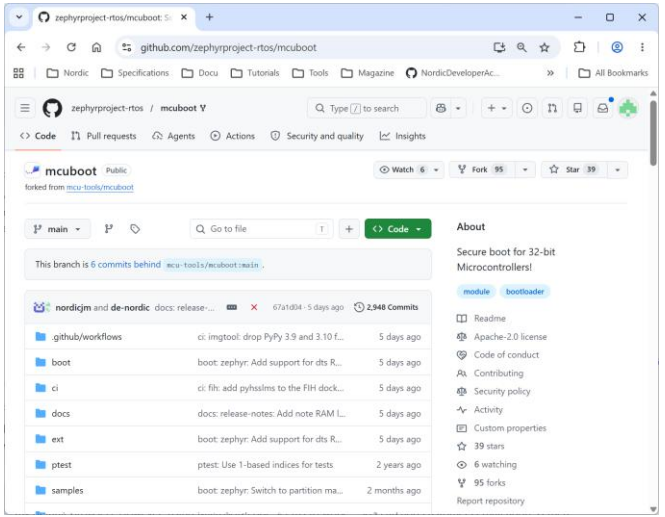
Zephyr Repositories (www.github.com/zephyrproject-rtos)







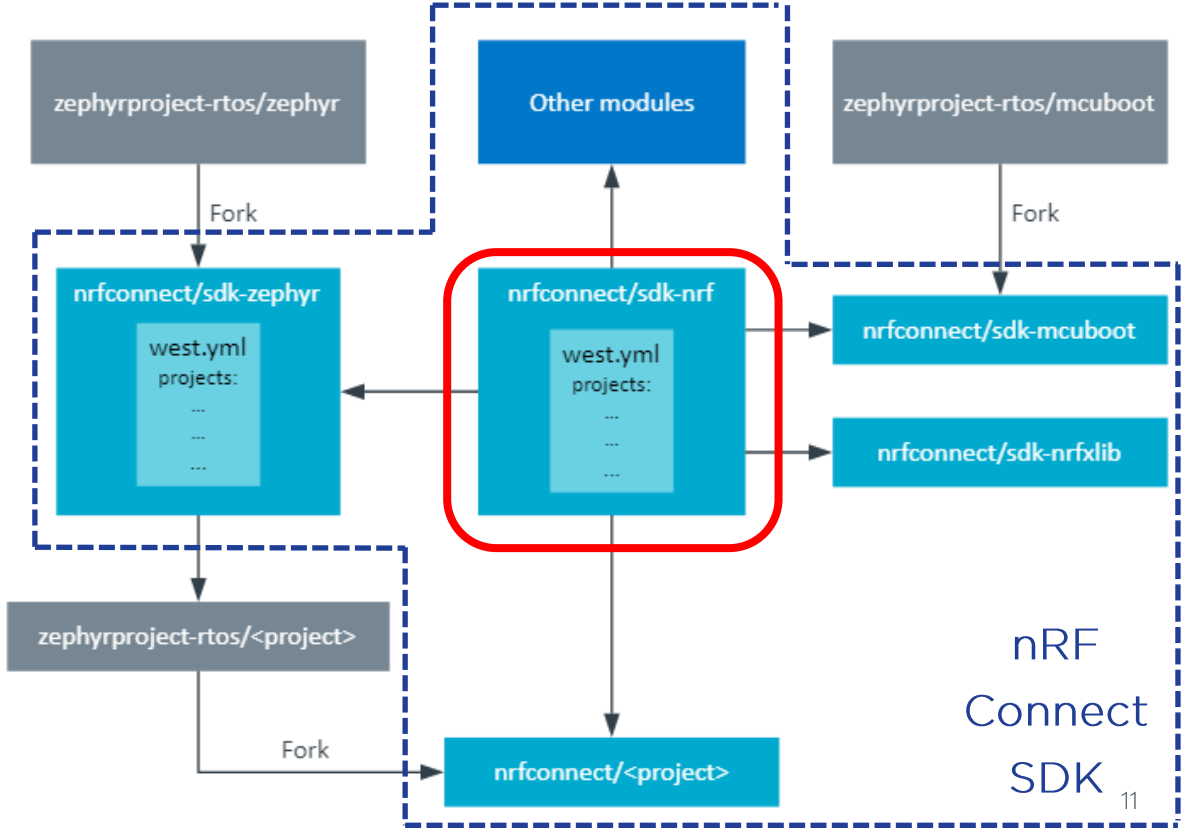




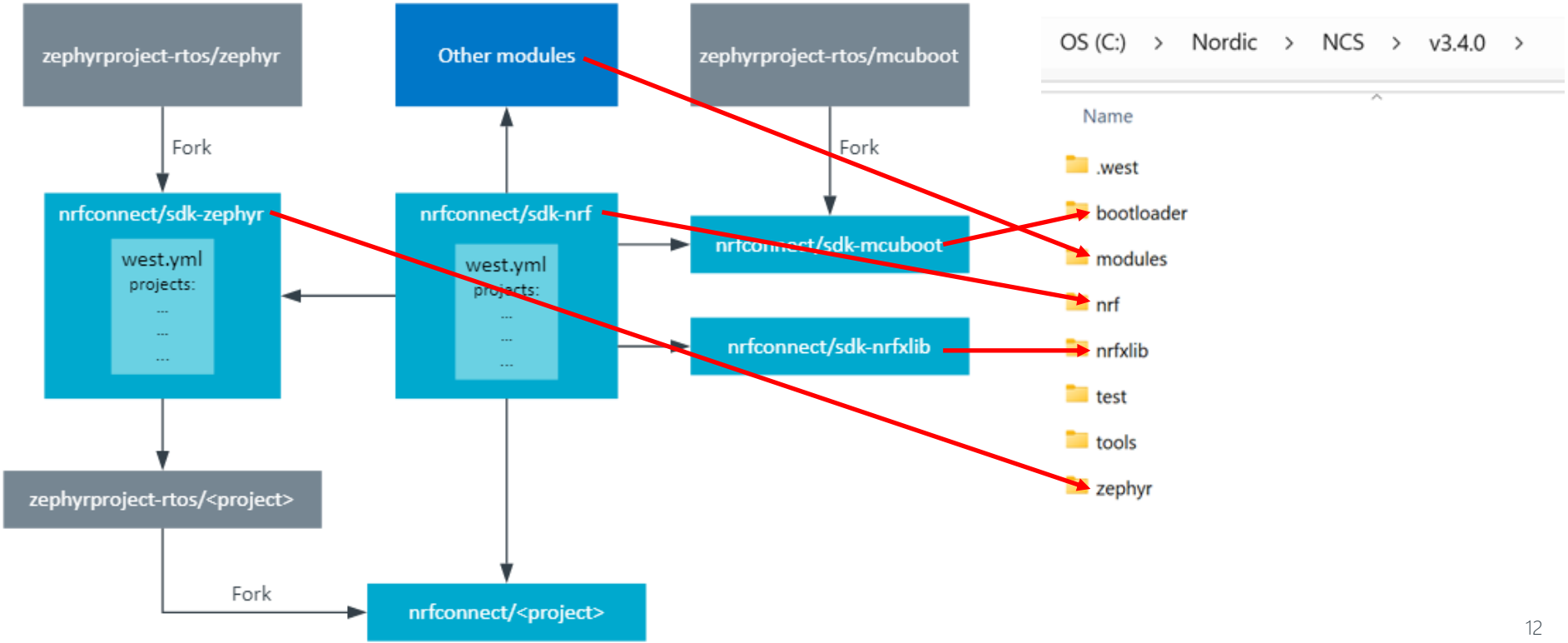
nRF Connect SDK Structure (www.github.com/nrfconnect)

- *nRF Connect SDK* is structured as star topology
  → **sdk-nrf** repo is the center
  (github.com/nrfconnect/sdk-nrf)

- **sdk-nrf** repo contains manifest file west.yml
  → Lists and points to all other repos included in the SDK



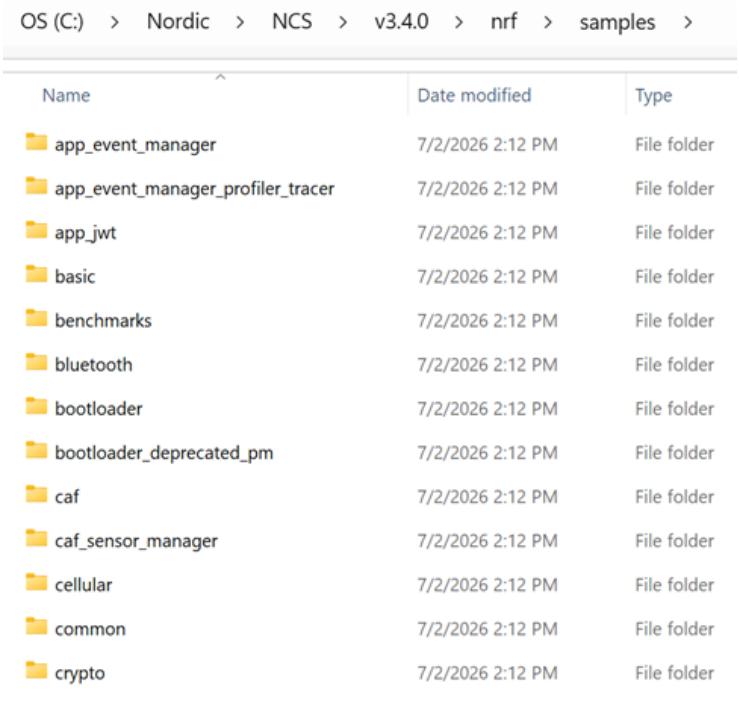
Repository Structure vs. Installation Paths



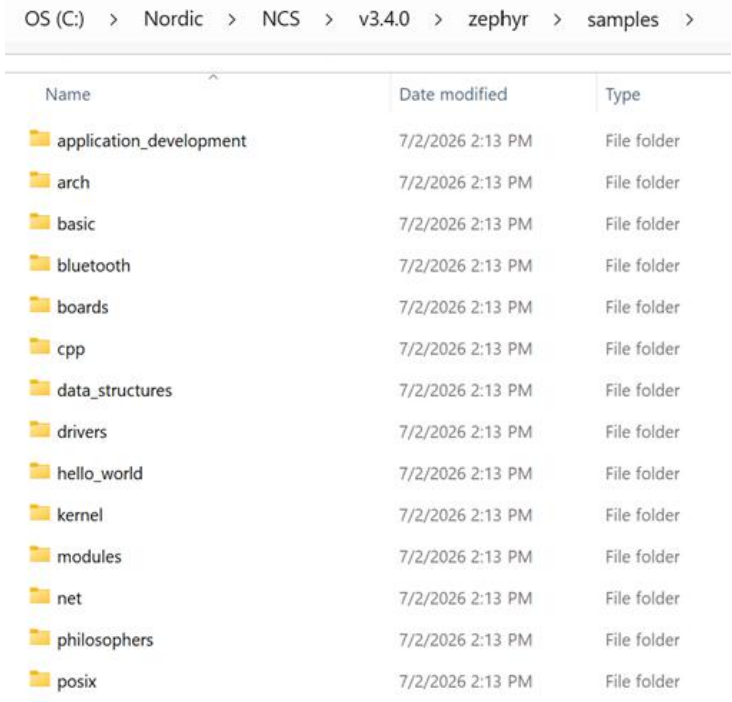
Code Examples

Code Examples can be found in different repositories, e.g. *nrf* or *zephyr*.

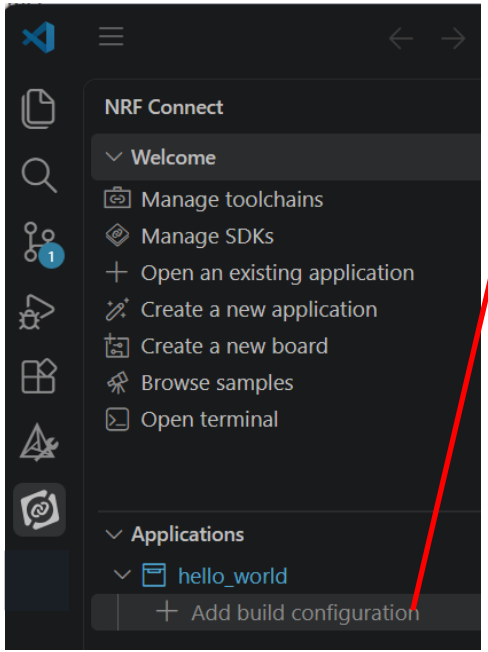
nrf repository => sample folder

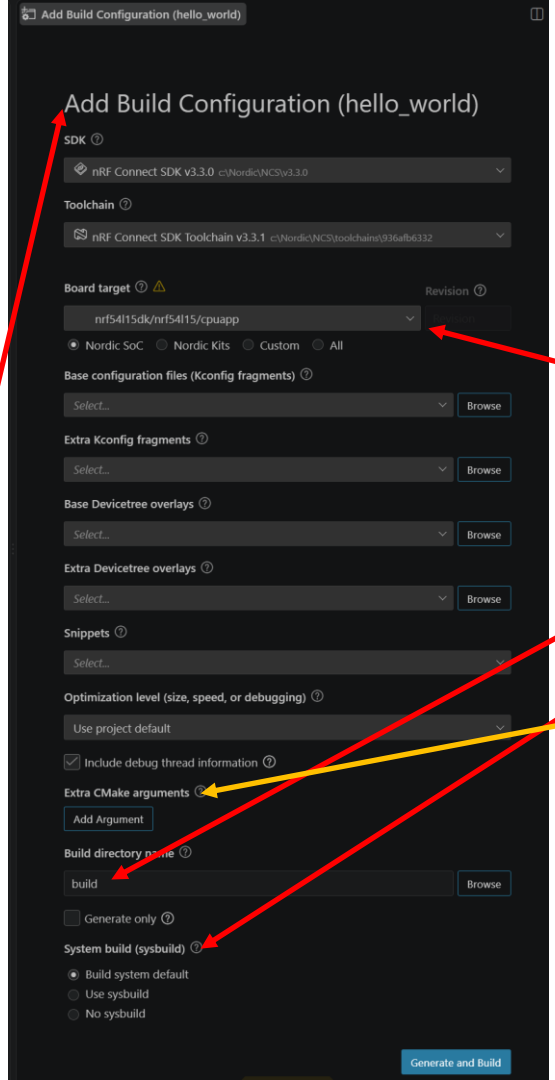


zephyr repository => sample folder



Command Line vs. GUI

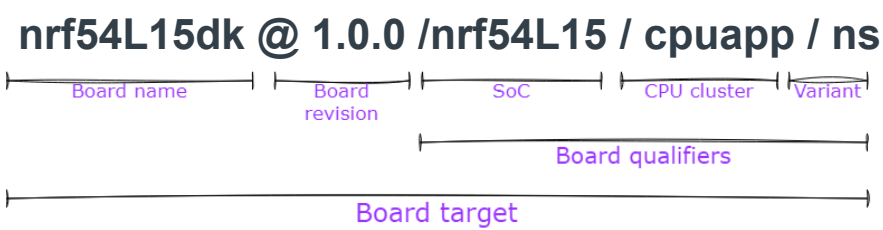


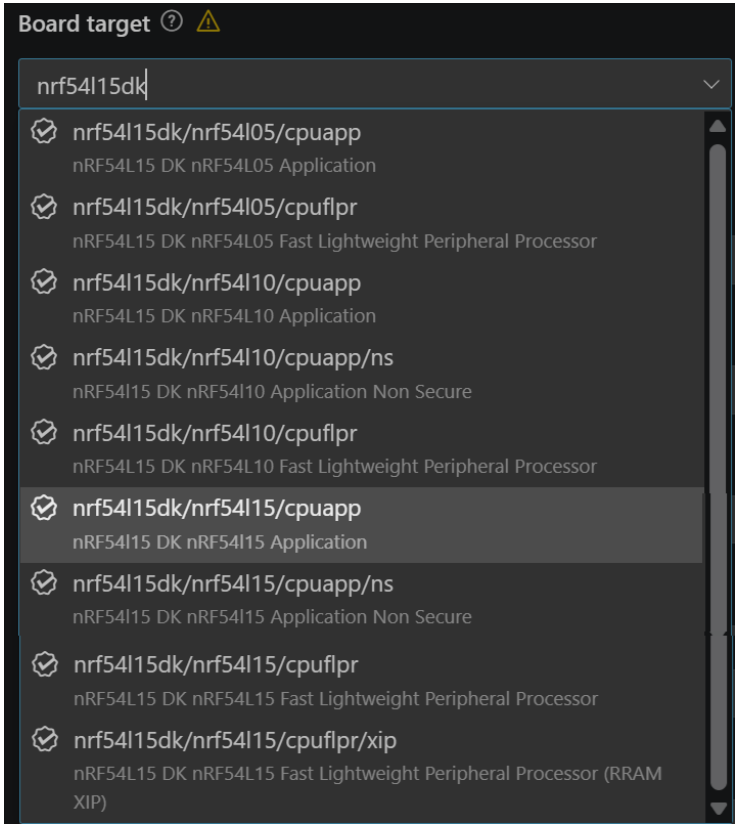



```
west build -b nrf54l15/nrf54l15/cpuapp
           --build-dir build
           --sysbuild
           --
           -DEXTRA_CONF_FILE=file.conf
```

Copyright © 2026 Nordic Semiconductor. All rights reserved

Board Terminology

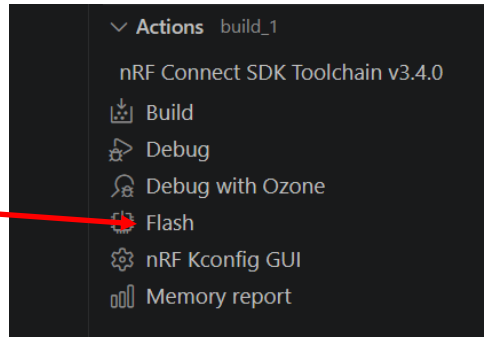




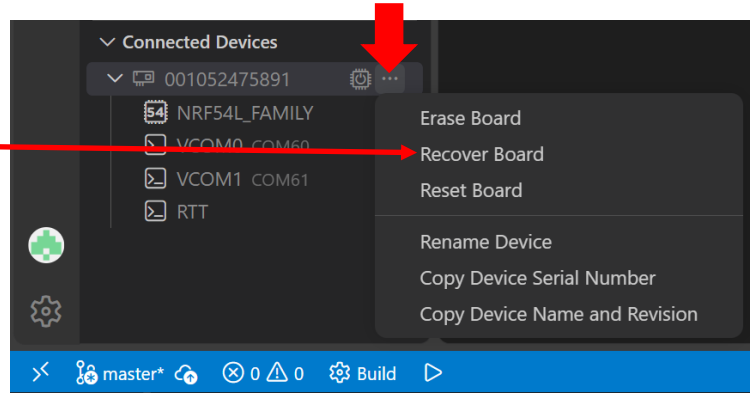
Copyright © 2025 Nordic Semiconductor. All rights reserved

Command Line vs. GUI

```
west flash
```



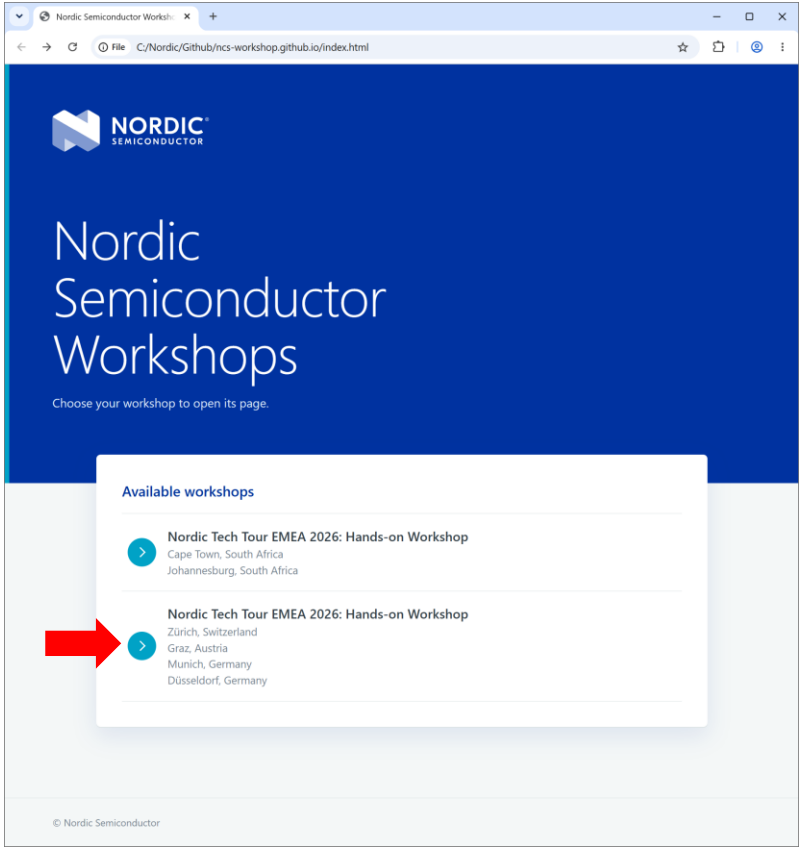

```
west flash --recover
```

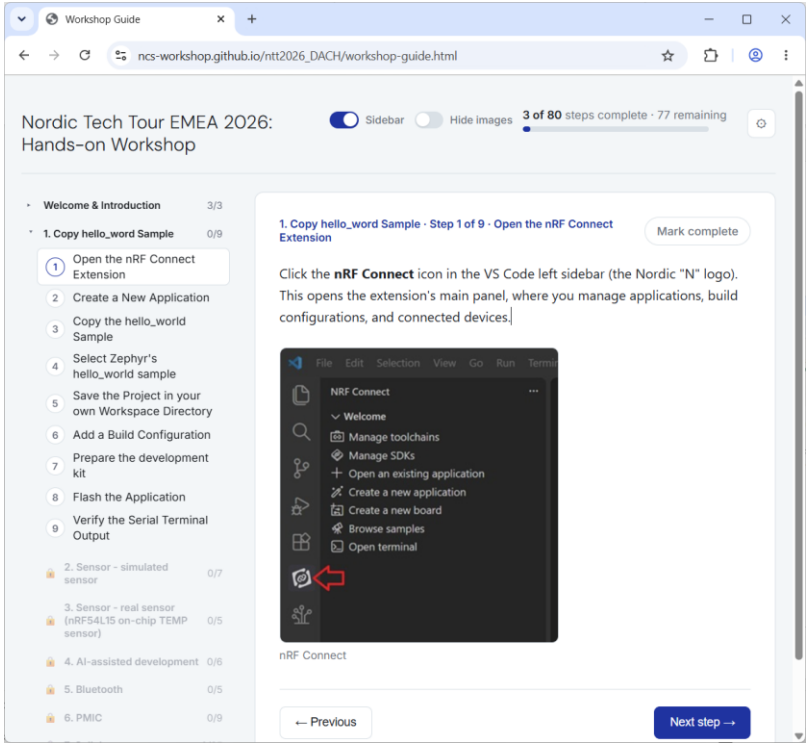


Copyright © 2026 Nordic Semiconductor. All rights reserved

Hands-on: Let's start with *hello_world*

You find the Workshop Guide here: http://ncs-workshop.github.io

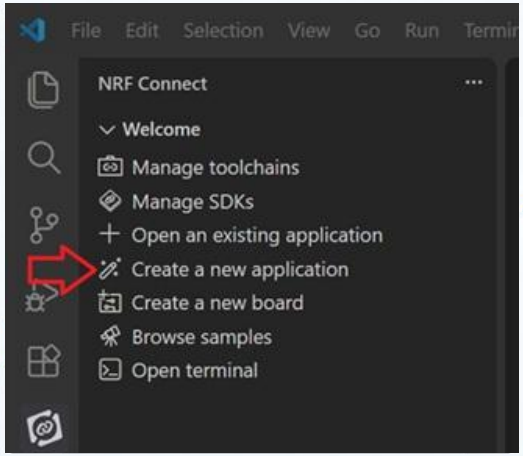




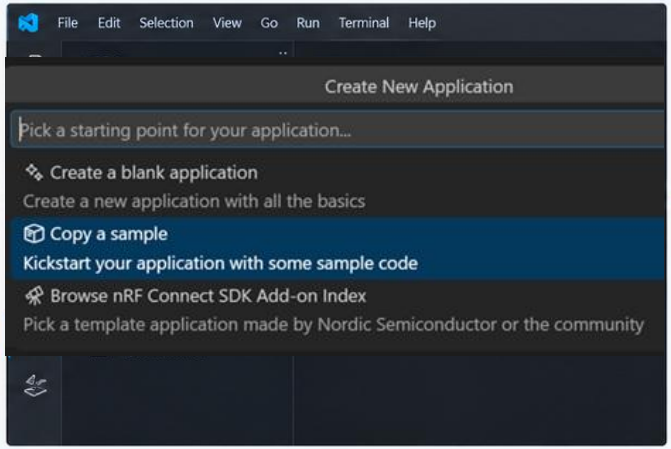
Copy a Sample in nRF Connect for VS Code

Create new application → Copy a sample → Select Zephyr's *hello world*

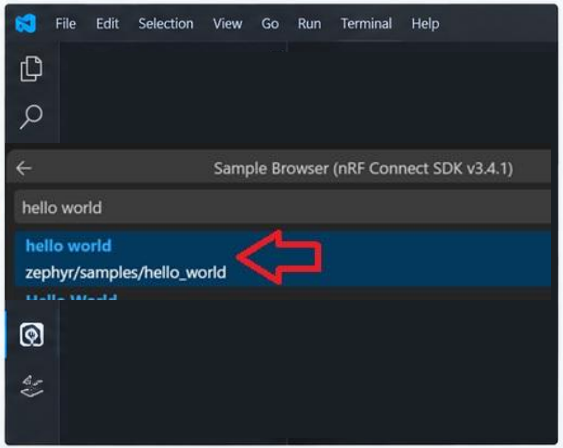
1 Open nRF Connect



2 Copy a sample



3 Select the Zephyr Hello World sample



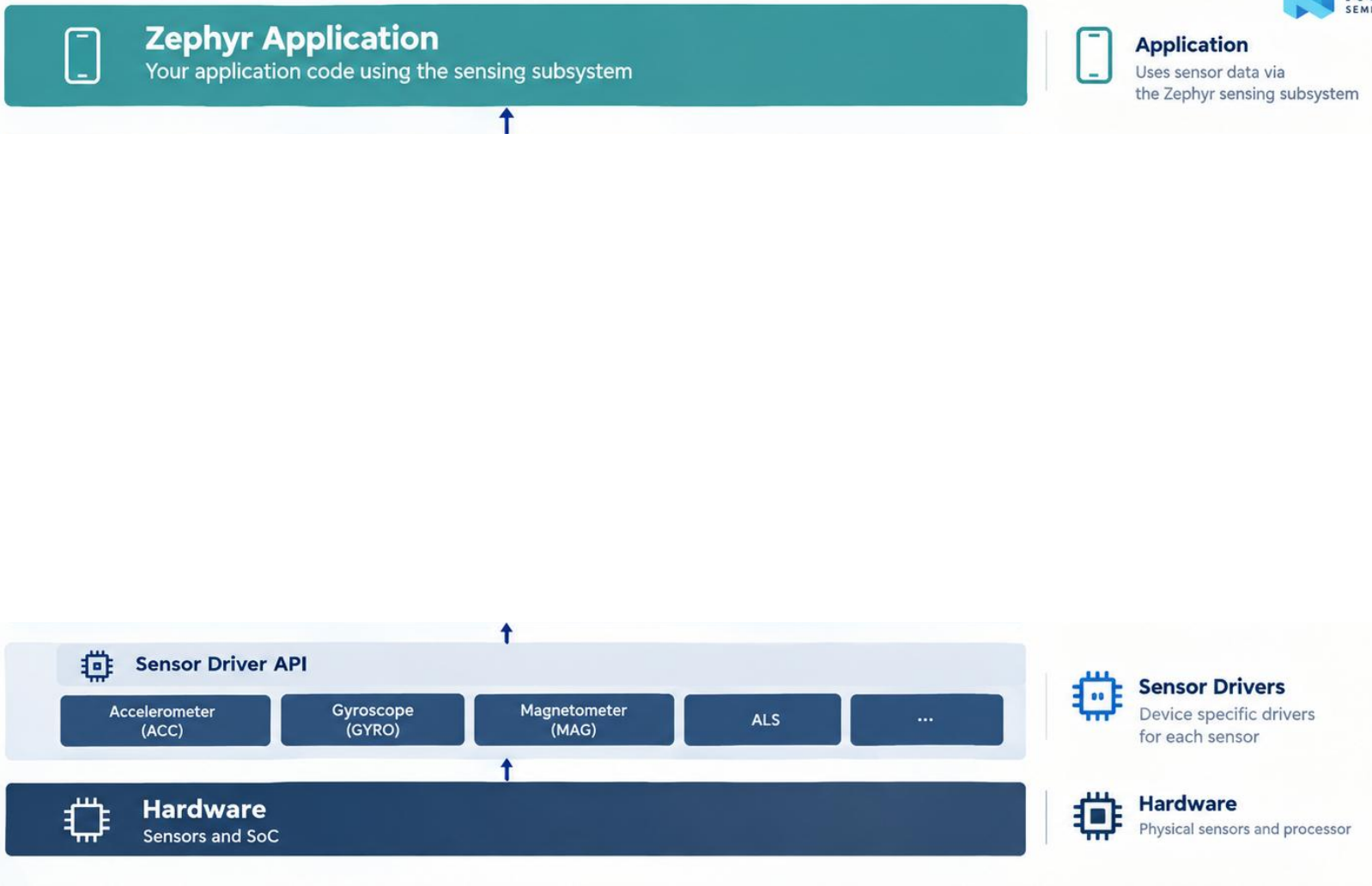
Copyright © 2026 Nordic Semiconductor. All rights reserved

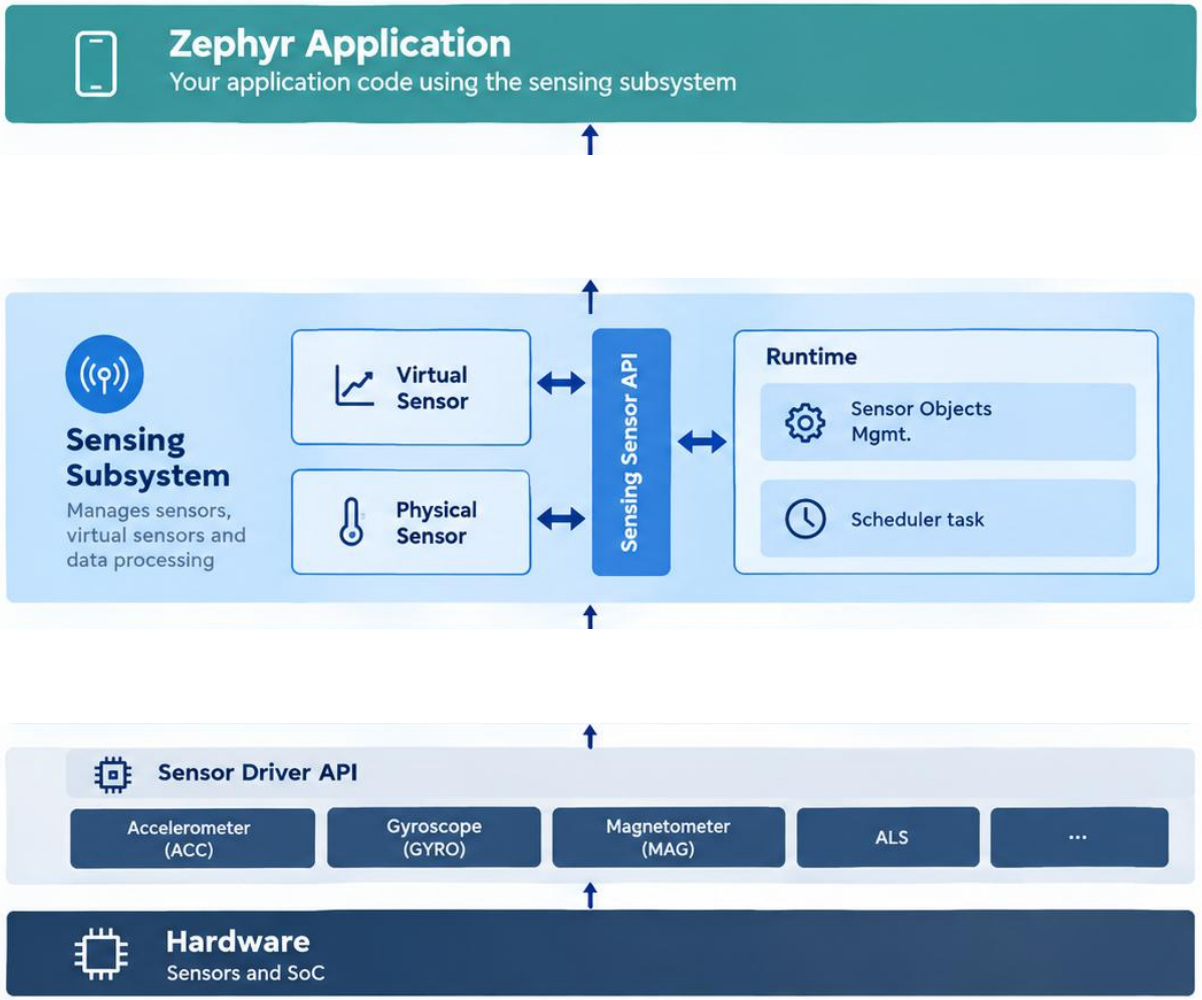
Sensing Subsystem

1 Sensor



Start with a simulated sensor to collect data.
At the end, replace it with a real sensor.


Zephyr Application
Your application code using the sensing subsystem
Application
Uses sensor data via
the Zephyr sensing subsystem
Sensor Driver API
Accelerometer
(ACC)
Gyroscope
(GYRO)
Magnetometer
(MAG)
ALS
...
Sensor Drivers
Device specific drivers
for each sensor
Hardware
Sensors and SoC
Hardware
Physical sensors and processor

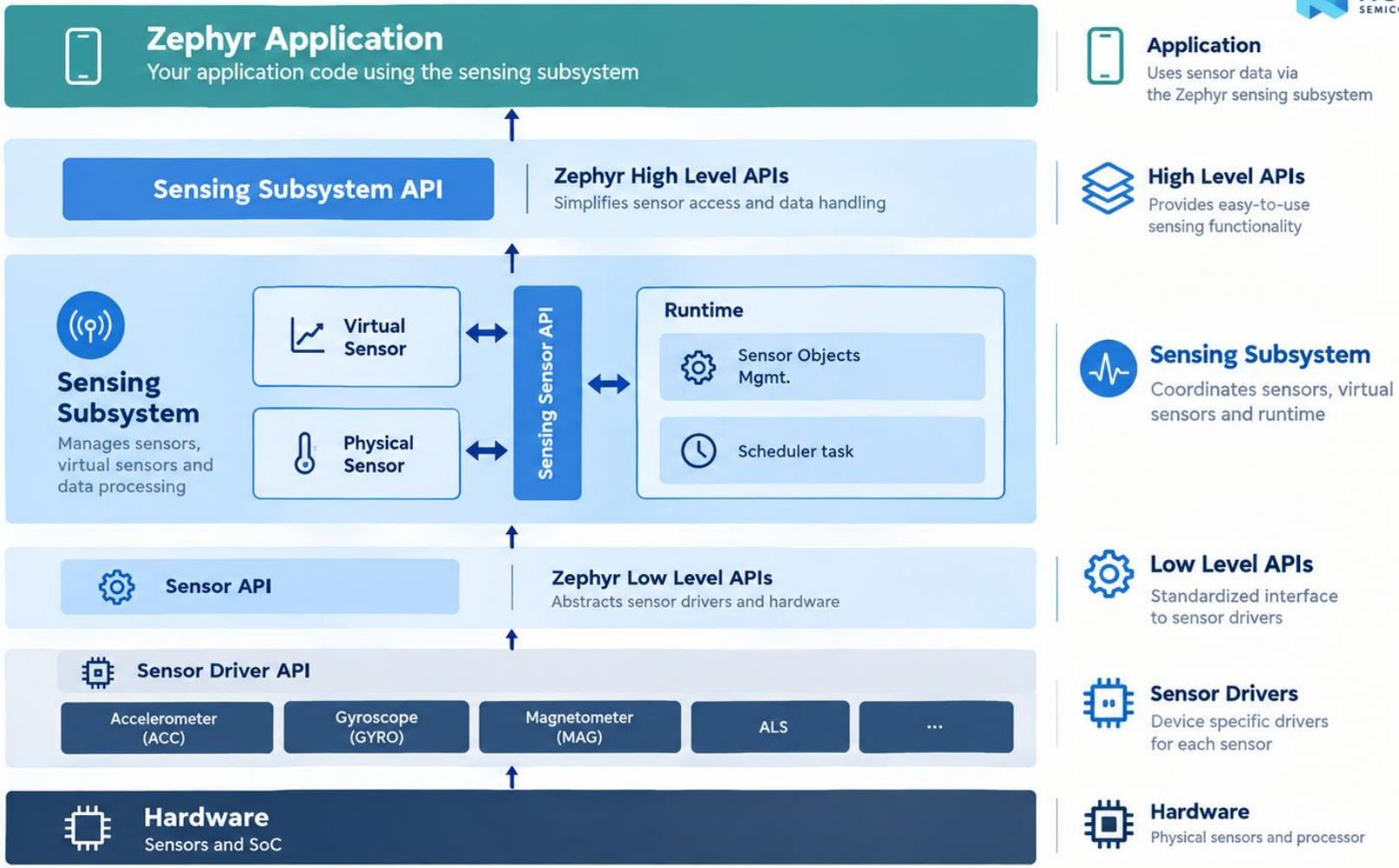


Application
Uses sensor data via the Zephyr sensing subsystem

Sensing Subsystem
Coordinates sensors, virtual sensors and runtime

Sensor Drivers
Device specific drivers for each sensor

Hardware
Physical sensors and processor


Zephyr Application
Your application code using the sensing subsystem
Sensing Subsystem API
Zephyr High Level APIs
Simplifies sensor access and data handling
Sensing Subsystem
Manages sensors, virtual sensors and data processing
Virtual Sensor
Physical Sensor
Sensing Sensor API
Runtime
Sensor Objects Mgmt.
Scheduler task
Sensor API
Zephyr Low Level APIs
Abstracts sensor drivers and hardware
Sensor Driver API
Accelerometer (ACC)
Gyroscope (GYRO)
Magnetometer (MAG)
ALS
...
Hardware
Sensors and SoC
Application
Uses sensor data via the Zephyr sensing subsystem
High Level APIs
Provides easy-to-use sensing functionality
Sensing Subsystem
Coordinates sensors, virtual sensors and runtime
Low Level APIs
Standardized interface to sensor drivers
Sensor Drivers
Device specific drivers for each sensor
Hardware
Physical sensors and processor

Zephyr Sensing Subsystem – High Level APIs

Simple and consistent APIs for sensor access and data handling

NORDIC SEMICONDUCTOR

Zephyr Application

Your application code using the sensing subsystem

Zephyr High Level APIs

Simplifies sensor access and data handling

sensor_sample_fetch()

Fetches the latest sample from a sensor and stores it in the sensor's data buffer.

DATA ACQUISITION

sensor_channel_get()

Retrieves the value of a specific sensor channel (e.g. temperature, acceleration).

DATA ACCESS

sensor_attr_get()

Reads a sensor attribute (e.g. range, resolution, power mode).

CONFIGURATION

sensor_attr_set()

Sets a sensor attribute (e.g. sampling rate, range, power mode).

CONFIGURATION

sensor_trigger_set()

Configures a trigger (e.g. data ready, threshold) and callback.

EVENTS & TRIGGERS

sensor_channel_get_all()

Retrieves multiple channel values in one call.

BATCH DATA



Hands-on: Add a simulated Sensor

KCONFIG
Enable Sensing Subsystem

prj.conf
```
CONFIG_SENSOR=y
```

DeviceTree
Enable simulated Sensor

app.overlay
```
simulated_sensor: sensor_sim{…}
```

Link in C Code
Using DT macros

main.c
```
#define SENSOR_NODE
  DT_NODELABEL(simulated_sensor)
 …
```

Device is running
Check if device is ok

main.c
```
if (!device_is_ready(dev)) {
        /* ERROR */
        return;
}
/* no error */
```

Using the Sensor

main.c
```
if (sensor_sample_fetch(dev) < 0)
{ /*ERROR*/ }
if (sensor_channel_get(dev,
       SENSOR_CHAN_AMBIENT_TEMP, &temp) < 0)
{ /*ERROR*/ }
```

Copyright © 2026 Nordic Semiconductor. All rights reserved



Hands-on: Replace simulated Sensor by TEMP

KCONFIG
Enable Sensing Subsystem

prj.conf

```
CONFIG_SENSOR=y
```

DeviceTree
Enable TEMP Sensor

app.overlay

```
aliases {
        die-temp0 = &temp;
};
```

Link in C Code
Using DT macros

Update needed to point to new hardware:

```
DT_ALIAS(die_temp0)
```

Device is running
Check if device is ok

main.c

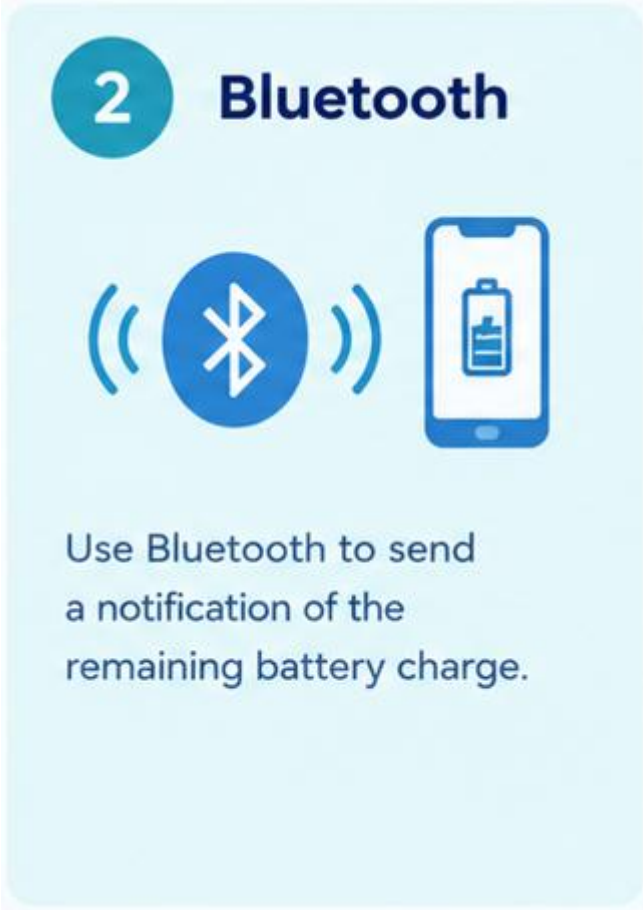
```
if (!device_is_ready(dev)) {
        /* ERROR */
        return;
}
/* no error */
```

Using the Sensor

Sensor_channel_get() requires update or channel (=> `SENSOR_CHAN_DIE_TEMP`)

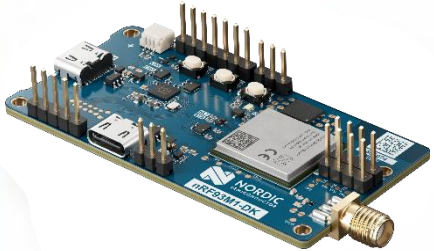
Copyright © 2026 Nordic Semiconductor. All rights reserved

nRF54L15 Bluetooth LE



Solution Overview

A Bluetooth LE application for the nRF93M1-DK using nRF Connect SDK v3.4.0



nRF93M1-DK
(nRF93M1QFCL v1A)

nRF93M1-DK (Peripheral)

Hello World

Bluetooth LE

GAP
- Advertising
- Connection

GATT
- Battery Service (BAS)
- Battery Level



Smartphone
(BLE Central)

Board
nRF93M1-DK

SDK
nRF Connect SDK

Bluetooth
LE Peripheral

Use AI assistance
in VS Code

Copyright © 2026 Nordic Semiconductor. All rights reserved

AI-assisted development: How it works

Adding context and Nordic knowledge to generic AI tools



Develop source code, configure and build projects, flash and debug hardware, monitor logs and outputs.

Whatever IDE and LLM your team is using today: VS Code, Cursor, Claude Code, Windsurf, terminal, etc.

Access to all Nordic technical documentation, including nRF Connect SDK plus DevAcademy, and DevZone Q&A.
nRF Connect SDK prompt templates.
Field data from nRF Cloud.

Copyright © 2026 Nordic Semiconductor. All rights reserved



Using the Nordic MCP server

Learn how: nordicsemi.com/aidev

MyNordic: https://mynordic.nordicsemi.com/en/sign-up/create-account

1. **Sign up for a myNordic account**
   Existing DevZone users should already have been transferred, new users can sign up on mynordic.nordicsemi.com

2. **Login to VS Code and point your assistant to the MCP(s) you want to use:**
   Nordic MCP: https://aidev.nordicsemi.com/mcp
   nRF Cloud MCP: https://app.memfault.com/mcp

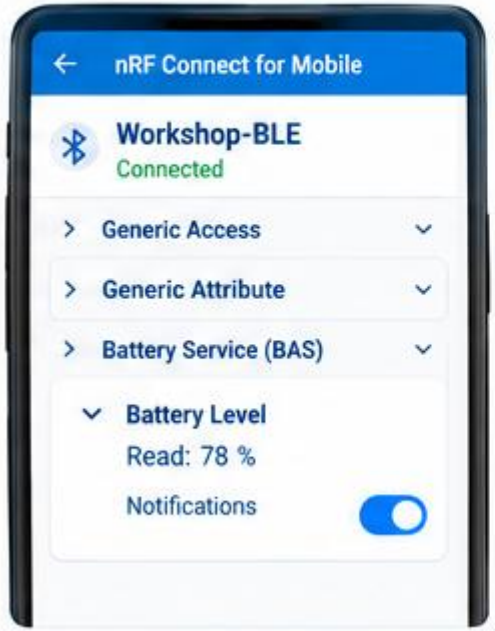
3. **Complete authentication**
   Usually completed from within the assistant's user interface

4. **Read the prompting guidelines and use the provided tools**
   Instruct your assistant to use the available tools, read the prompting guidelines to learn how to prompt efficiently



Copyright © 2026 Nordic Semiconductor. All rights reserved

Final Result



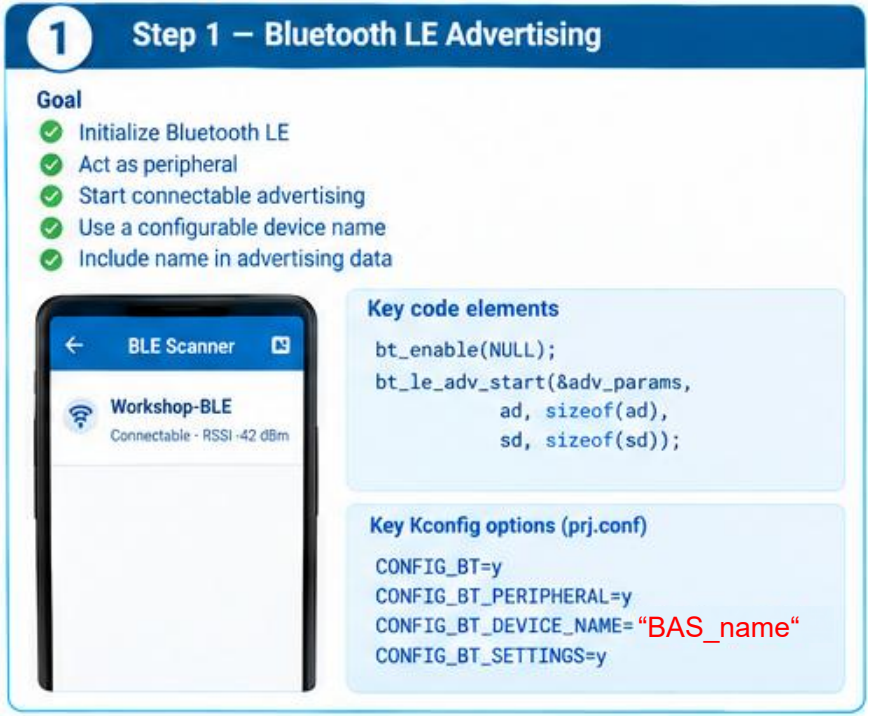
Behavior

1. Advertise with configurable name („BAS_workshop“)
2. Establish a connection (log messages)
3. Provide Battery Service (BAS) with simulated battery level (100 → 0 , every 1 s)

The smartphone can read the battery level and receive notifications.

Copyright © 2026 Nordic Semiconductor. All rights reserved





PROMPT 1:

Use the Nordic MCP server to add Bluetooth initialization and advertising to my existing project.

Please:

1. **Initialize the Bluetooth stack** using the appropriate APIs and configuration for this project and its target board.

2. Check and **handle the initialization result** before proceeding.

3. Only after successful Bluetooth initialization, **start Bluetooth LE advertising**.

4. Set the Bluetooth advertising/**device name** to <ENTER YOUR BT ADV NAME HERE>. The name must start with BAS_.

5. Use the existing project structure and nRF Connect SDK conventions. Do not introduce unnecessary changes to unrelated code.

6. **Update any required Kconfig** or project configuration so that the Bluetooth functionality is enabled.

7. Clearly identify all files and configuration changes you make.

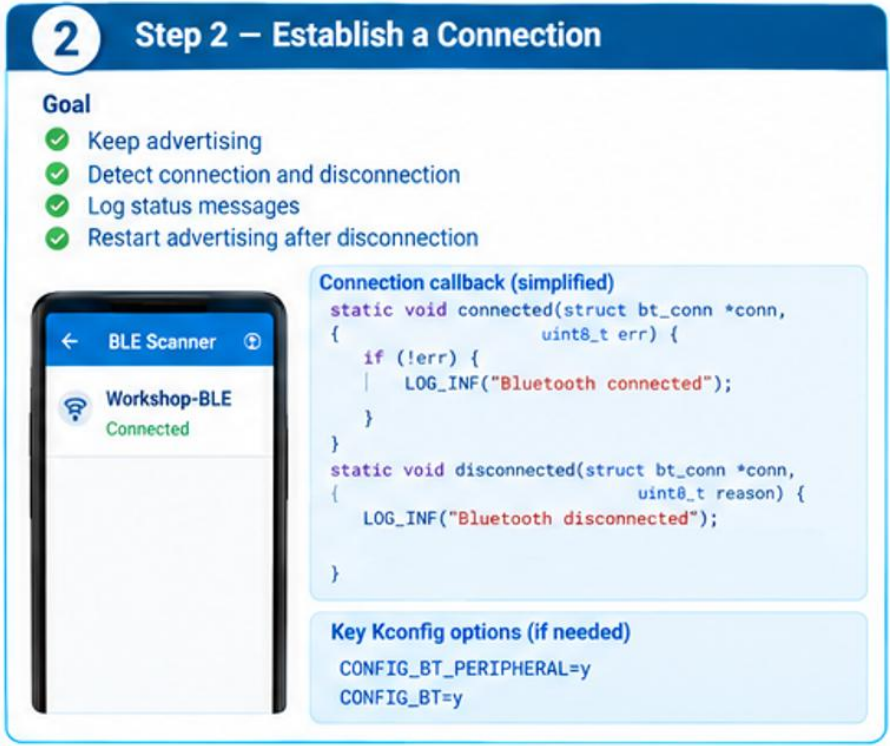
8. **All Bluetooth functions should be placed in the file bluetooth.c.** Create a header file bluetooth.h that declares a function that declares the bluetooth_init() function..

Use the Nordic MCP documentation and tools to verify that the implementation is appropriate for the project's nRF Connect SDK version and target board.

Copyright © 2026 Nordic Semiconductor. All rights reserved

PROMPT 2:





Use Nordic MCP.

Extend the existing Bluetooth LE implementation to include connection establishment and the Battery Service (BAS).

The Bluetooth stack is already initialized and connectable advertising is already running. Do not reinitialize the Bluetooth stack, recreate advertising, or replace existing Bluetooth functionality.

First inspect the existing project to determine the Bluetooth SDK/framework and follow its existing APIs and architecture.

Requirements:

- Add Bluetooth **connection establishment**.

- **Handle connection success and failure** through the existing Bluetooth event/callback mechanism.

- Add the standard Bluetooth SIG **Battery Service (BAS)** to the existing GATT database.

- Add the **Battery Level characteristic** with read and notification support.

- Provide an **application API to update the battery level (0–100%).**

- In the main loop, a variable named “battery_level” should be used, whose value is **decremented from 100 to 0** and which is used for BAS notifications.

- **Send Battery Level notifications** when the connected client has enabled them.

Copyright © 2026 Nordic Semiconductor. All rights reserved

PROMPT 2 - Continued:





- Preserve the existing initialization, advertising, connection, and application architecture.
- Do not invent APIs; use APIs available in the detected SDK/version.
- Build the project after implementation and fix compilation errors and warnings introduced by the changes.
- Add all Bluetooth **code to the bluetooth.c** file.
- **Handle a disconnect and restarting of advertising**.

After implementation, briefly summarize:

- Modified files.
- Connection establishment behavior.
- BAS implementation.
- Battery Level update API.
- Build/validation results.

Copyright © 2026 Nordic Semiconductor. All rights reserved

nPM1300 Power Management

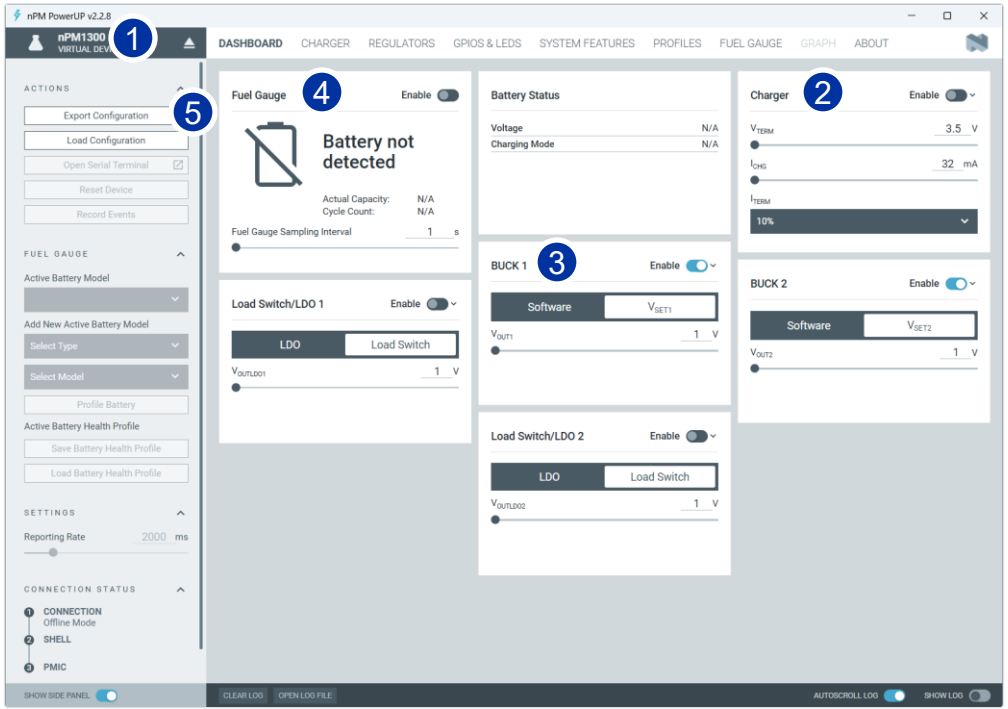
3 PMIC



Use the PMIC for power supply and to measure battery parameters (voltage, current, temperature).

Report state of charge and state of health using the Nordic fuel gauge algorithm.

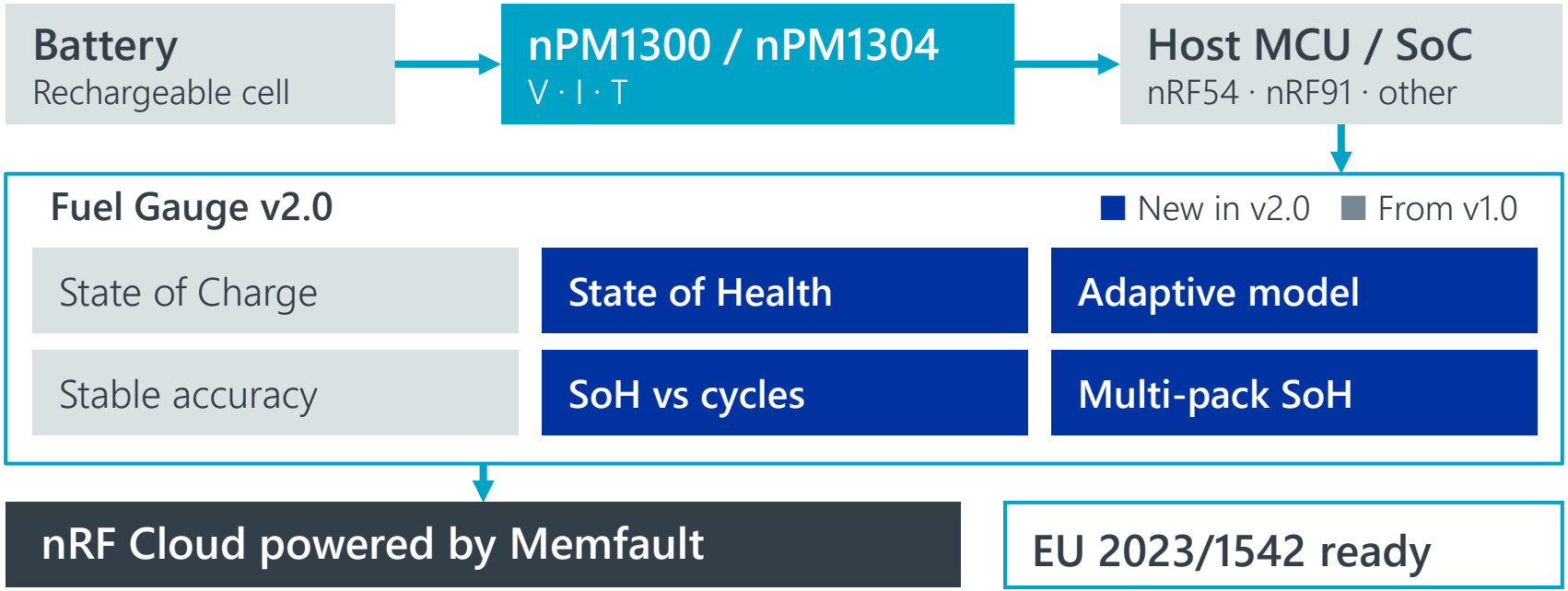
Configure the nPM1300 with nPM PowerUp



1. **Connect**
   EK over USB, or virtual device
2. **Charger**
   Set VTERM, ICHG, ITERM
3. **Regulators**
   Set BUCK and LDO voltages
4. **Fuel gauge**
   Pick battery model, enable
5. **Export**
   Save as devicetree overlay

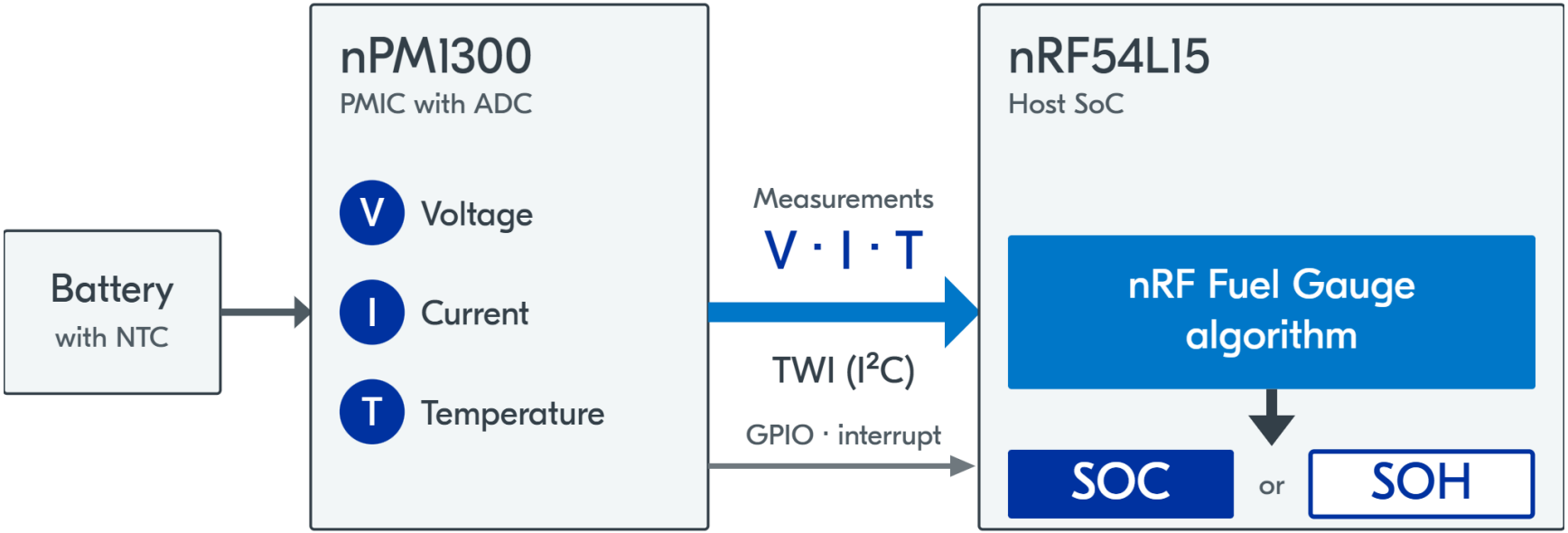
Nordic Fuel Gauge v2.0

Battery intelligence in software for nPM1300 and nPM1304



Copyright © 2026 Nordic Semiconductor. All rights reserved

nPM1300 feeds the fuel gauge on the nRF54L15



The nRF54L15 reads V, I and T from the nPM1300 and calculates the battery state.

nRF Fuel Gauge API

All functions start with nrf_fuel_gauge_

1 · Setup

init	Start library
config_params_default_get	Load defaults
config_params_current_get	Read current
config_params_adjust	Apply changes

2 · Run

process	Feed V, I, T
ext_state_update	Report events
idle_set	Enter idle

3 · Results

soc_get	State of charge
soh_get	State of health
tte_get	Time to empty
ttf_get	Time to full

4 · Save state

state_get	Read state
state_compatible_check	Check version

Using init and process

Call init once, then process for every new measurement

nrf_fuel_gauge_init(&params, &v0)

Call once at start

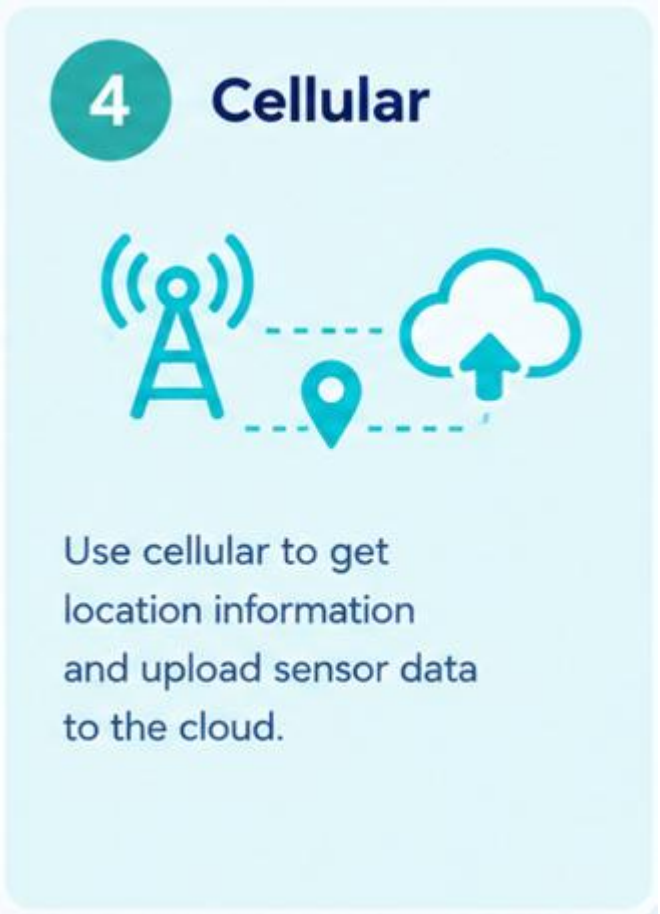
v0	Initial voltage [V]
i0	Initial current [A]
t0	Initial temperature [°C]
model	Battery model
opt_params	Tuning (or NULL)
state	Saved state (or NULL)

nrf_fuel_gauge_process(v, i, T, t_delta, &soc, &state)

Call for every new measurement

v	Voltage [V]
i	Current [A]
T	Temperature [°C]
t_delta	Time since last call [s]
soc	Out: charge [%]
state	Out: debug info (or NULL)

nRF93M1 & LTE Cat 1 bis

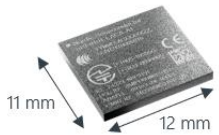


Comparison at a glance

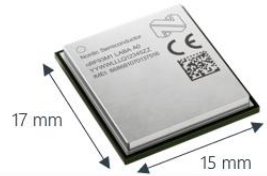
Typical values for each technology

	NB-IoT	LTE-M	LTE Cat 1 bis
Data rate	~250 kbps	up to 1 Mbps	10 / 5 Mbps (down/up)
Bandwidth	200 kHz	1.4 MHz	up to 20 MHz
Mobility	Fixed use	Handover	Full mobility
Power use	Lowest	Low	Medium
Typical use	Meters, parking	Asset trackers	Telematics, kiosks

nRF9151 LTE-M, NB-IoT and NTN



nRF93M1 LTE Cat 1 bis



Copyright © 2026 Nordic Semiconductor. All rights reserved

Two ways to use the modem

A AT commands via serial terminal

Bypass-modem sample

Type AT commands live

Fast to try, no code to write

B Modem library on nRF54L15

App runs on the nRF54L15

Library handles the modem

Closer to a real product

Copyright © 2026 Nordic Semiconductor. All rights reserved

Anatomy of a JWT

A JSON Web Token (JWT) is a compact, URL-safe token consisting of three parts:



Header

Defines the token type and signing algorithm.

```
{
    "alg": "ES256",
    "typ": "JWT"
}
```

Base64URL encoded

Payload

Contains the claims (e.g. identity and metadata).

```
{
    "sub": "device-123",
    "iat": 1793430000,
    "exp": 1793433600
}
```

Base64URL encoded

Signature

Verifies the token's integrity using a private key.

Created with:

ES256 (ECDSA with P-256 and SHA-256)

Common JWT Claims Explained

alg	typ	sub	iat	exp
Signing algorithm (e.g. ES256)	Token type (e.g. JWT)	Subject / device identity	Issued At (token creation timestamp)	Expiration Time (token expiry timestamp)

Key Takeaways

- ✓ A JWT has 3 parts: Header, Payload, Signature
- ✓ It is signed (e.g. with ES256)
- ✓ Treat JWTs as sensitive (e.g. bearer tokens)

Does “device time” need to be set?

For provisioning (AT%REGJWT):

You do **not** need to set the device system time. The registration JWT used during onboarding can be generated without a valid system time.

For post-provisioning authentication (AT%JWT):

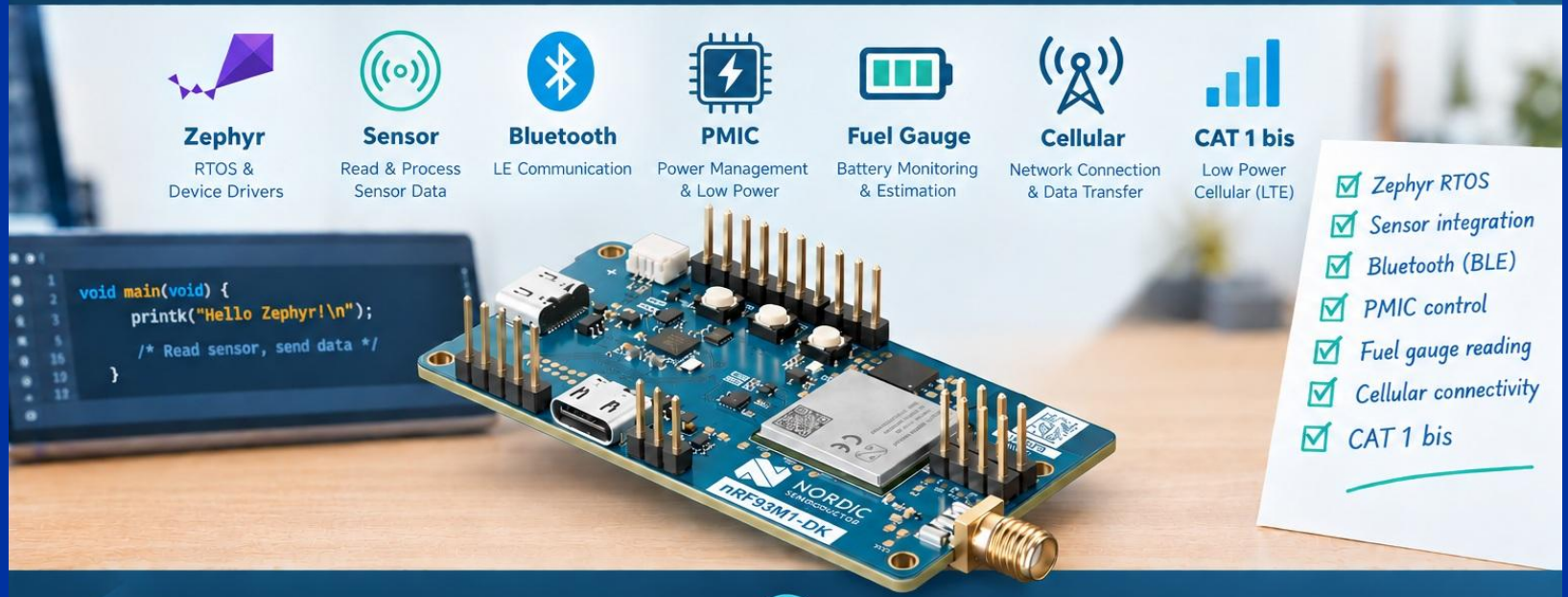
You do need a valid system time. The short-lived authentication JWT (used after onboarding to access cloud APIs) will be rejected by nRF Cloud if the device time is not set.

Copyright © 2026 Nordic Semiconductor. All rights reserved

Wrap-up

WORKSHOP HANDS-ON COURSE

Successfully Completed



NORDIC SEMICONDUCTOR

Course Completed!

Real Devices.
Live Data.
Practical Skills.